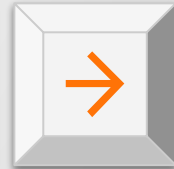




Capa de aplicación HTTP



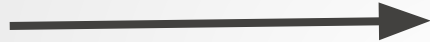
Características de HTTP 1.0, 1.1 y 2.0



Ignacio Traberg

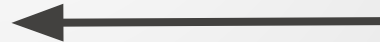
Estructura de request/response

Request



```
GET /index.html HTTP/1.1  
Host: www.ejemplo.com  
Connection: keep-alive
```

```
HTTP/1.1 200 OK  
Content-Type: text/html  
Content-Length: 1024
```



Response

HTTP 1.0 vs 1.1

HTTP/1.0 (1996):

- Connection: **close** por defecto (*no persistente*)
- Host: opcional
- Una conexión por objeto

HTTP/1.1 (1997):

- Connection: **keep alive** por defecto (*persistente*)
- Host: obligatorio
- Soporta chunked transfer

HTTP 2.0

HTTP/2.0 (2015)

- **Binario** en lugar de texto plano
- Multiplexación: varias requests/responses en **paralelo**
- Compresión de headers (HPACK): menos repetición, menos overhead.
- Priorización de streams: Varios streams a la vez, con priorización por orden o urgencia (html > css)
- Negociación en TLS (ALPN): elección segura del protocolo

GET, POST y HEAD

GET

- **Cuerpo vacío** (datos en la URL)
- Cacheable, idempotente
- Menos seguro

POST

- El cuerpo lleva datos
- **No cacheable**, usado para cambios
- Más **seguro** para datos sensibles

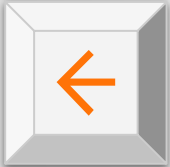
HEAD

- Igual a GET pero sin cuerpo en la respuesta (**solo headers**)
- Útil para ver si un recurso **existe** o bien su **tamaño**

Códigos comunes:

200 OK – 301 Moved – 404 Not Found – 500 Internal Error

Los **headers** más utilizados



Host: servidor al que se envía la request (obligatorio en 1.1)

User-Agent: identifica al cliente que envía el request

Connection: *keep-alive* o *close*

Content-Type: tipo de dato enviado (*text/html*, *application/json*, etc.)

Content-Length: tamaño del body en bytes

Authorization: credenciales para autenticación

Accept: formatos que el cliente acepta (*text/html*, *application/json*, etc.)

Cache-Control: indica políticas de cacheo

Cookie / Set-Cookie: manejo de estado y sesiones

If-Modified-Since / ETag: validación de recursos en caché.

Demo GET 1.0

```
$ curl -v --http1.0 http://www.example.com/
* Host www.example.com:80 was resolved.
* IPv6: (none)
* IPv4: 181.30.145.8, 181.30.145.35
* Trying 181.30.145.8:80...
* Connected to www.example.com (181.30.145.8) port 80
* using HTTP/1.x
> GET / HTTP/1.0
> Host: www.example.com
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent off
* HTTP 1.0, assume close after body
```

```
< HTTP/1.0 200 OK
< Content-Type: text/html
< ETag:
"84238dfc8092e5d9c0dac8ef93371a07:1736799080.121134"
< Last-Modified: Mon, 13 Jan 2025 20:11:20 GMT
< Cache-Control: max-age=2896
< Date: Thu, 28 Aug 2025 20:10:53 GMT
< Content-Length: 1256
< Connection: close
< X-N: S
<
<!doctype html>
<html>
<head>
<title>Example Domain</title>
```

Demo GET 1.1

```
$ curl -v --http1.1 http://www.example.com/
* Host www.example.com:80 was resolved.
* IPv6: (none)
* IPv4: 181.30.145.35, 181.30.145.8
* Trying 181.30.145.35:80...
* Connected to www.example.com (181.30.145.35) port 80
* using HTTP/1.x
> GET / HTTP/1.1
> Host: www.example.com
> User-Agent: curl/8.14.1
> Accept: */*
>
* Request completely sent off
```

```
< HTTP/1.1 200 OK
< Content-Type: text/html
< ETag:
"84238dfc8092e5d9c0dac8ef93371a07:1736799080.121134"
< Last-Modified: Mon, 13 Jan 2025 20:11:20 GMT
< Cache-Control: max-age=2497
< Date: Thu, 28 Aug 2025 20:17:32 GMT
< Content-Length: 1256
< Connection: keep-alive
<
<!doctype html>
<html>
<head>
  <title>Example Domain</title>
```


Demo GET 2.0

```
$ curl -v --http2 https://www.google.com/
* Trying 172.217.28.4:443...
* Connected to www.google.com (172.217.28.4) port 443 (#0)
* ALPN, offering h2
* SSL connection using TLSv1.3 /
  TLS_AES_256_GCM_SHA384
* ALPN, server accepted to use h2
* Server certificate:
* subject: CN=www.google.com
* issuer: Google Trust Services
* valid: Aug 11 2025 to Nov 3 2025
* SSL certificate verify ok.
* Using HTTP2, server supports multiplexing
* Connection state changed (HTTP/2 confirmed)

> GET / HTTP/2
> Host: www.google.com
> user-agent: curl/7.81.0
> accept: */*

< HTTP/2 200
< date: Thu, 28 Aug 2025 21:11:35 GMT
< cache-control: private, max-age=0
< content-type: text/html; charset=ISO-8859-1
< server: gws
< x-frame-options: SAMEORIGIN
< alt-svc: h3=":443"; ma=2592000,h3-29=":443"; ma=2592000

<!doctype html><html itemscope
itemtype="http://schema.org/WebPage" ...
```

- ALPN (Application-Layer Protocol Negotiation): Durante el **handshake** se acuerda usar **h2** (http 2.0).
- SSL (Secure Sockets Layer): Protocolo de seguridad deprecado.
- TLS (Transport Layer Security): Evolución de SSL, más seguro y actualizado, usado hoy en HTTPS.